

BLE-MIRABILIS-BLUE

Firmware Developer Guide

GATT Interface, Pairing, Notifications & File Transfer Protocol

Firmware v0.6.2

Purpose

This guide is written for mobile, desktop and embedded developers integrating with BLE-MIRABILIS-BLUE. It describes the hardware baseline, GATT interface, security model, characteristic-level operations, and the bidirectional file-transfer protocol. It is intended to be sufficient for implementing a client without reading the firmware source code.

Tutorial scope: BLE fundamentals through secure bidirectional file transfer. GPIO and DFU are intentionally out of scope.

Contents

- 1. Hardware overview
- 2. Firmware overview
- 3. BLE architecture and connection model
- 4. GATT services and characteristics
- 5. Developer operations
- 6. Pairing, bonding and encrypted access
- 7. File transfer protocol
- 8. File storage, limits and lifecycle
- 9. Testing and integrity verification
- 10. Integration guidance and error handling
- 11. Quick reference
- Appendix A. Protocol constants
- Appendix B. Sources and implementation basis

1. Hardware overview

BLE-MIRABILIS-BLUE runs on a nice!nano-compatible nRF52840 board. The firmware uses the board as a Bluetooth Low Energy peripheral and exposes a compact GATT interface designed specifically for developer education.

Item	Developer-relevant specification
SoC	Nordic Semiconductor nRF52840
CPU	64 MHz Arm Cortex-M4 with floating-point unit
Memory	1 MB internal flash, 256 KB RAM
Wireless	2.4 GHz radio with Bluetooth Low Energy support
USB	USB 2.0 device capability on the nRF52840; the tutorial board is powered/flushed over USB
Tutorial device name	BLE-MIRABILIS-BLUE
Firmware revision	0.6.2
Hardware revision string	NICE-NANO-COMPATIBLE
Application storage model	Single temporary file slot in RAM; maximum 16 KiB

Important distinction

The nRF52840 itself has 1 MB of flash, but BLE-MIRABILIS-BLUE v0.6.2 intentionally stores transferred file data in RAM. The 16 KiB file limit is therefore an application-level design choice, not the physical flash capacity of the SoC.

1.1 Boot and power behavior

The tutorial board is normally powered through its USB connection. Removing USB power causes the application to stop and the RAM-backed file to be lost. A firmware reset likewise clears application RAM. A BLE disconnect by itself does not clear the file.

2. Firmware overview

The firmware is a progressive BLE teaching peripheral. It exposes standard Device Information characteristics plus a custom service that demonstrates increasingly advanced GATT operations.

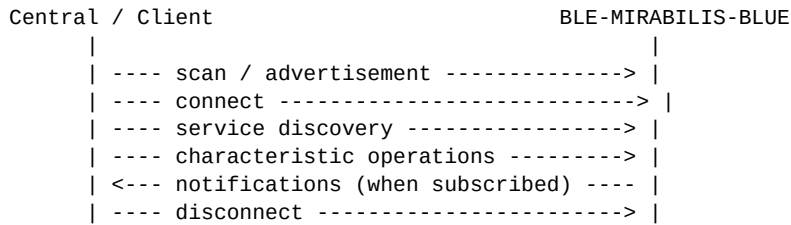
- Standard GATT reads for serial number, hardware revision and firmware revision.
- Write with response followed by read-back.
- Write driving a separate Read + Notify state characteristic.
- A Notify-only periodic event stream.
- Write Without Response followed by read-back.
- Pairing/bonding-backed encrypted characteristics.
- Secure bidirectional file transfer using Write Without Response + Notify.
- A secure cumulative Total Uploaded Bytes metric.

Not included in this guide

The experimental File Count characteristic (...000D) is intentionally omitted from the developer-facing interface described here. GPIO and DFU are also outside the tutorial scope.

3. BLE architecture and connection model

BLE-MIRABILIS-BLUE acts as a GATT server / BLE peripheral. A phone, Mac, tablet or other BLE central scans for the device, connects, discovers services and characteristics, and performs operations according to each characteristic's properties.



3.1 Service model

Service	UUID	Purpose
Device Information Service	0x180A	Standard identity and revision strings.
BLE-MIRABILIS-BLUE Custom Service	7E57A000-0000-4B1A-9C00-000000000001	Tutorial operations, security examples and file transfer.

4. GATT services and characteristics

4.1 Device Information Service

Characteristic	UUID	Properties	Access	Meaning
Serial Number String	0x2A25	READ	Open	Uppercase hexadecimal string derived from the nRF52840 hardware device ID.
Hardware Revision String	0x2A27	READ	Open	NICE-NANO-COMPATIBLE
Firmware Revision String	0x2A26	READ	Open	0.6.2

4.2 Custom service

Name	UUID suffix	Properties	Security	Purpose
Basic Write	...0002	WRITE	Open	Store arbitrary bytes.
Last Written Value	...0003	READ	Open	Read latest ...0002 value; N/A before first write.
Observable Write	...0004	WRITE	Open	Change the value exposed by ...0005.
Observable Value	...0005	READ NOTIFY	Open	Read current state and receive notifications after ...0004 changes it.
Periodic Event Stream	...0006	NOTIFY	Open	One-byte sequential event every 2 seconds after subscription.
Write Without	...0007	WRITE NO RSP	Open	Write using ATT Write

Response				Command.
Last WNR Value	...0008	READ	Open	Read latest ...0007 value; N/A before first write.
Secure Write	...0009	WRITE	Encrypted	Secure write example.
Secure State	...000A	READ NOTIFY	Encrypted	Secure read/notification state driven by ...0009.
File Transfer RX	...000B	WRITE NO RSP	Encrypted	Central -> peripheral file-transfer channel.
File Transfer TX	...000C	NOTIFY	Encrypted CCCD	Peripheral -> central file-transfer channel.
Total Uploaded Bytes	...000E	READ	Encrypted	uint64 little-endian cumulative successful-upload bytes since boot.

5. Developer operations

All custom UUIDs use the prefix 7E57A000-0000-4B1A-9C00-0000000000xx.

5.1 Basic WRITE -> READ

1. WRITE bytes to ...0002 using Write With Response.
2. READ ...0003.
3. The returned bytes match the most recent successful write.

Example:

Client -> ...0002: DE AD BE EF

Client <- ...0003: DE AD BE EF

5.2 WRITE -> READ + NOTIFY

1. Subscribe to notifications on ...0005.
2. WRITE a new value to ...0004.
3. The peripheral updates its observable state.
4. Subscribers receive the new value on ...0005.
5. A READ of ...0005 returns the same current value.

This pattern models a command/state pair commonly used by connected devices.

5.3 Notify-only event stream

1. Subscribe to ...0006.
2. No READ is required or supported.
3. While subscribed, the peripheral emits a one-byte sequential value every 2 seconds.

Example stream:

00, 01, 02, 03, ... FF, 00, 01 ...

This demonstrates asynchronous peripheral-generated events that are not responses to a client read.

5.4 WRITE WITHOUT RESPONSE -> READ

1. Send bytes to ...0007 using Write Without Response.
2. READ ...0008.
3. ...0008 returns the latest value received by ...0007.

Unlike ...0002, the write operation does not generate an ATT write response.

5.5 Secure WRITE -> READ + NOTIFY

1. Connect to the peripheral.
2. Subscribe/read ...000A or write ...0009.
3. Because these attributes require encryption, the central must establish a secure link.
4. WRITE bytes to ...0009.
5. ...000A is updated and notifies subscribed clients.
6. READ ...000A to retrieve the current secure state.

5.6 Read Total Uploaded Bytes

READ ...000E after an encrypted connection is established.

The characteristic returns an unsigned 64-bit little-endian value representing the cumulative number of payload bytes accepted by successful uploads since the current boot.

Example: 12,288 decimal = 0x00000000000003000
On-air little-endian bytes: 00 30 00 00 00 00 00 00

6. Pairing, bonding and encrypted access

Characteristics ...0009, ...000A, ...000B, ...000C and ...000E are protected by encrypted GATT permissions. The central therefore cannot use them as ordinary open characteristics.

Characteristic	Protected operation
...0009	Encrypted WRITE
...000A	Encrypted READ and encrypted notification subscription (CCCD)
...000B	Encrypted WRITE WITHOUT RESPONSE
...000C	Encrypted notification subscription (CCCD)
...000E	Encrypted READ

6.1 Expected security flow

```

Connect
|
Attempt secure operation
|
Central establishes encryption / pairing as required
|
Bond may be retained
|
Secure GATT operation succeeds

```

6.2 Bond-state mismatch

Developer note

During testing, forgetting BLE-MIRABILIS-BLUE on the central while the peripheral still retained its bond could leave the peers with mismatched security state. The symptom may be an 'insufficient encryption' error rather than a fresh pairing prompt. Resetting/restarting the peripheral and clearing the stale central bond restored normal pairing behavior. Client applications should treat this as a recoverable security-state problem, not as a file-transfer protocol failure.

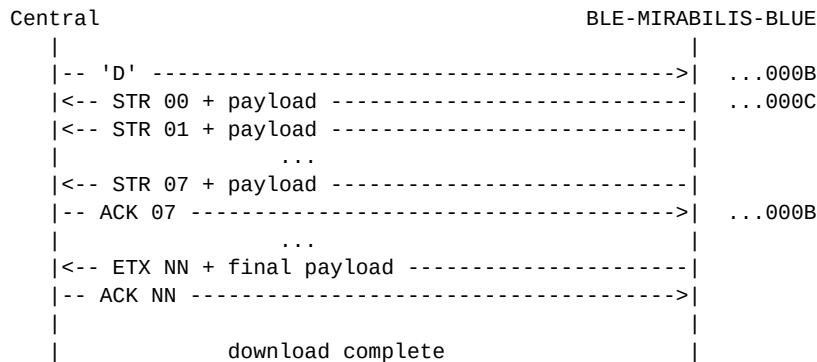
Direction	Characteristic	GATT operation	Content
Central -> Peripheral	...000B FILE_TRANSFER_RX	WRITE WITHOUT RESPONSE	Upload request/chunks; download ACK/NACK/CAN.
Peripheral -> Central	...000C FILE_TRANSFER_TX	NOTIFY	Download chunks; upload ACK/NACK.

Value	Name	Meaning
0x02	STR	Normal data chunk.
0x03	ETX	Final data chunk of the complete transfer.
0x06	ACK	Batch accepted; second byte identifies the acknowledged sequence.
0x15	NACK	Receiver reports a batch/chunk problem.
0x18	CAN	Abort/cancel transfer.
0x55 ('U')	UPLOAD	Start upload.
0x44 ('D')	DOWNLOAD	Start download of the currently stored file.

Developer integration reference - tutorial firmware

The upload request used by the supplied Python tester is byte 'U' followed by a 32-bit little-endian file size. The sender then transmits 18-byte payload chunks on ...000B. After each group of up to eight chunks, it waits for ACK/NACK on ...000C.

7.4 Download - peripheral to central



The central must subscribe to ...000C before issuing the download command. Downloaded payload bytes are concatenated in sequence order. The central sends ACKs on ...000B after every eight chunks or immediately after the ETX packet.

7.5 Reliability rules

- ACK identifies the last successfully accepted sequence number.
- NACK reports a missing, invalid or otherwise unacceptable batch/chunk.
- CAN may be used to abort a transfer.
- A BLE disconnect aborts an in-progress transfer.
- Sequence-number rollover is valid and must not be treated as corruption.
- The tutorial uses eight chunks per acknowledgement, matching the production-style reference protocol on which the exercise was modeled.

8. File storage, limits and lifecycle

Maximum file size: 16 KiB (16,384 bytes)

BLE-MIRABILIS-BLUE v0.6.2 uses a fixed 16 KiB RAM buffer for the current file. A client must not assume that larger uploads can be retained. The file-size ceiling is an application limit of this tutorial firmware.

Event	Current file	Total Uploaded Bytes
Successful first upload	Stored in RAM	Increases by uploaded payload size
Successful later upload	Previous file is replaced	Increases again
BLE disconnect	Retained	Retained
BLE reconnect	Retained	Retained
Download	Retained; download is non-destructive	Unchanged
Repeated download	Retained	Unchanged
Firmware reset	Lost	Resets with application state
USB cable removed / power loss	Lost	Resets on next boot

8.1 Why the file is temporary

The tutorial intentionally keeps file data in RAM instead of implementing a filesystem or flash-backed persistence layer. This keeps the example focused on BLE transport behavior: chunking, sequencing, acknowledgements, notifications, security and reconstruction.

8.2 Oversize files

The firmware buffer is 16 KiB. Client code should validate the file size before starting an upload and reject files larger than 16,384 bytes. Do not design a production client around undefined behavior beyond this limit.

9. Testing and integrity verification

9.1 Python environment

```
python3 -m venv .venv
source .venv/bin/activate
pip install bleak
```

9.2 Create a binary test file

```
head -c 8192 /dev/urandom > demo_upload.bin
```

9.3 Run the transfer test

```
python3 ble_mirabilis_file_transfer_test.py
```

The supplied tester scans for BLE-MIRABILIS-BLUE, connects, subscribes to ...000C, uploads demo_upload.bin, downloads the stored file, and writes the result to downloaded.bin.

9.4 Verify byte-for-byte integrity

```
cmp demo_upload.bin downloaded.bin

# or:
shasum -a 256 demo_upload.bin downloaded.bin
```

Expected result

cmp prints nothing when the files are identical. For SHA-256 verification, both files must produce the same digest. The tutorial implementation has been validated with random 4 KiB and 8 KiB binary files, including sequence-number rollover.

9.5 Metadata read

A separate metadata monitor can READ ...000E after pairing/encryption to verify that cumulative successful-upload bytes increase as expected.

10. Integration guidance and error handling

- Discover the custom service/characteristics by UUID; do not make application correctness depend only on the advertised device name.
- Subscribe to a notification characteristic before issuing an operation whose response arrives asynchronously on that characteristic.
- Use Write With Response only for characteristics that expose WRITE; use Write Without Response for ...0007 and file-transfer RX ...000B.

- Establish/allow BLE security before using encrypted characteristics.
- Treat a disconnect during file transfer as an aborted operation.
- Implement sequence rollover correctly.
- Validate the 16 KiB maximum before upload.
- Implement timeouts around ACK/NACK waits in production clients.
- Do not assume the transferred file survives reset or power loss.
- For application-level integrity, compare expected size and/or a cryptographic hash after reconstruction when appropriate.

10.1 Suggested client architecture

```

BLE transport
|
+-- connection / discovery
+-- security / pairing state
+-- characteristic read/write/notify wrappers
|
File-transfer protocol
|
+-- chunk encoder / decoder
+-- sequence handling
+-- batch ACK/NACK handling
+-- timeout / cancellation
|
Application
|
+-- upload progress
+-- download reconstruction
+-- integrity verification

```

11. Quick reference

Operation	Characteristic(s)	Security	Client action
Read device info	2A25 / 2A27 / 2A26	Open	READ
Basic write/read	...0002 / ...0003	Open	WRITE, then READ
Observable state	...0004 / ...0005	Open	Subscribe ...0005, WRITE ...0004, optional READ ...0005
Periodic events	...0006	Open	Subscribe
WNR/readback	...0007 / ...0008	Open	WRITE NO RSP, then READ
Secure state	...0009 / ...000A	Encrypted	Pair/encrypt; WRITE / READ / subscribe
Upload	...000B / ...000C	Encrypted	Subscribe TX; send U + chunks on RX; process ACK/NACK
Download	...000B / ...000C	Encrypted	Subscribe TX; send D on RX; receive chunks; ACK on RX
Transfer metric	...000E	Encrypted	READ uint64LE

Appendix A. Protocol constants

Symbol	Value	Notes
STR	0x02	Normal chunk marker

ETX	0x03	Final chunk marker
ACK	0x06	Acknowledgement
NACK	0x15	Negative acknowledgement
CAN	0x18	Cancel/abort
UPLOAD	0x55 ('U')	Upload command
DOWNLOAD	0x44 ('D')	Download command
CHUNK_SIZE	20 bytes	2-byte header + up to 18-byte payload
PAYLOAD_SIZE	18 bytes	Maximum data per chunk
CHUNKS_PER_ACK	8	Maximum chunks before acknowledgement
MAX_FILE_SIZE	16,384 bytes	Single RAM-backed file slot

Appendix B. Sources and implementation basis

This guide is based primarily on the BLE-MIRABILIS-BLUE v0.6.2 GATT reference, the working Python/Bleak test client, and the GameGolf file-transfer protocol document supplied for architectural comparison. Hardware specifications are cross-checked against Nordic Semiconductor's nRF52840 documentation.

- BLE-MIRABILIS-BLUE GATT Table & Protocol Reference - Firmware v0.6.2.
- BLE-MIRABILIS-BLUE File Transfer Test (Python/Bleak) and accompanying README.
- GameGolf / Medallion File Transfer Protocol document: STR/ETX chunking, 18-byte payload, 8 chunks per ACK, ACK/NACK/CAN, valid uint8 sequence rollover and abort-on-disconnect behavior.
- Nordic Semiconductor nRF52840 product documentation: 64 MHz Arm Cortex-M4 with FPU, 1 MB flash, 256 KB RAM, Bluetooth LE and USB capability.

The BLE-MIRABILIS-BLUE file-transfer command header and storage behavior described here follow the tutorial firmware/test implementation, not the product-specific GameGolf data-type headers.